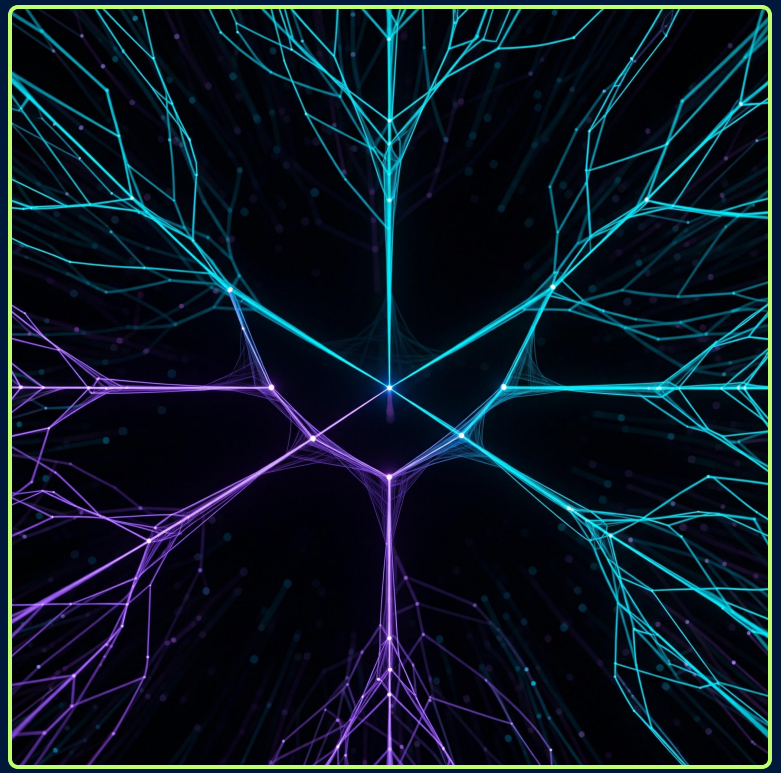
WELCOME BACK TO ALGO II

"From ALGO I to ALGO II — a quick wake-up, then we're back"



Welcome to **Algo & Data Structures II** . Last semester you built the foundation — arrays, linked lists, stacks, queues, hash tables, graphs, basic sorts.

You can now think in terms of **time and space cost** , not just "does it work?".

This semester takes you to the next level: **non-linear structures** (trees, BSTs, advanced graphs) and the **algorithmic paradigms** that exploit them — divide & conquer, recursion, dynamic programming.

Algo II = the structures that branch (trees, graphs) and the techniques that scale (D&C, DP). One semester to make **`O(n log n)`** your default reflex.

"Last semester you learned to walk. This semester you learn to climb."

▪ HOW THIS COURSE WORKS ▪

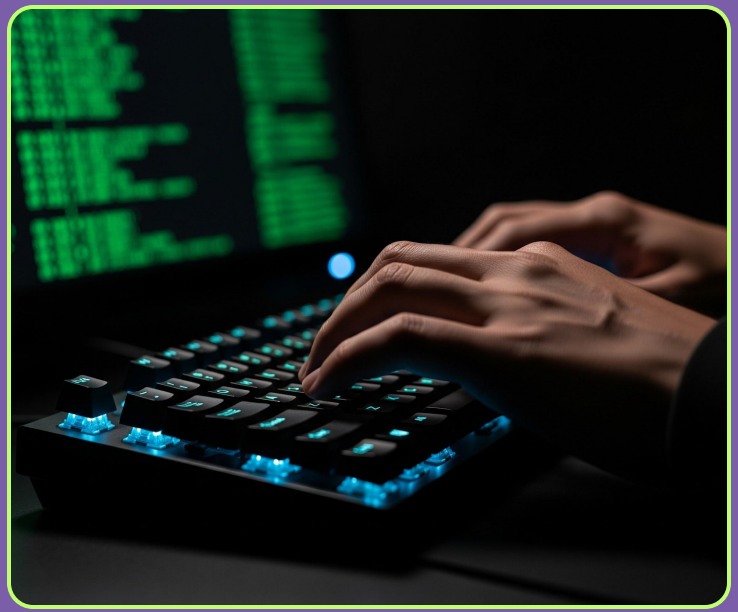
Algorithms reward practice more than reading. This course is built around typing code , not just listening — every class has a long hands-on block where you implement, test, and debug in Python. Before each class, you'll find everything you need on the course repo.

Access the course repository:

`https://gitee.com/adnan-boudjelal/algo_data-II`

THE SETUP PHASE

Today starts a short setup phase — philosophy, repo, key vocabulary. Once it's done, we move into real algorithms next class and stay there for the rest of the semester.



"Today we set the rules. Soon every class is fingers on the keyboard."

PRACTICE OVER THEORY

The lecture exists to prepare you for the exercises. Not the other way around. The only way coding sticks is by **typing it yourself**, again and again.

Rule	What it means
Always try	Even if you didn't read the notes. Open the terminal first.
Stuck = good	"Stuck" means your brain is working. Push through.
Finish the first exercises	Last ones can be hard (bonus). First ones cover the most important concept.

You don't learn algorithms by listening. You don't learn them by reading either.

You learn them by typing them out and watching them break.

"Read less. Type more. he students who finish confident with trees and DP are the ones who wrote the most code."

AI — TUTOR, NOT SOLVER

AI is a powerful tool. But power without skill is dangerous. An AI engineer who can't think without AI is not an engineer — they are a passenger. Right now, you are training to become the engineer.

✅ AI as a tutor

"I tried this. Give me a hint, not the answer. Explain this concept like I'm new to it."

Like a good teacher. Asks you questions, corrects you, makes you think.

❌ AI as a solver

"Solve this exercise for me. Give me the working code."

Useful for engineers shipping products.

Dangerous for students — it skips the part where your brain builds the skill.

"Use AI to learn faster. Never use it to skip learning."

WHAT YOU LOSE WHEN AI SOLVES

Every time you ask AI for the answer, something quiet happens. You feel productive. You move on. But the skill never builds. Here is what the shortcut actually costs you.

What happens	What you lose
You skip the struggle	Your brain doesn't build the skill
You copy-paste code	You can't reproduce it later, alone
You feel productive	You stay weak long-term
Habit of "asking AI"	You can't function without it

Struggling is the workout.

When you push through a hard exercise, your brain builds new connections. Skip the workout = no muscle.

"The discomfort of being stuck is the sound of your brain getting stronger."

▪ HOW TO TALK TO AI ▪

The same tool helps you or hurts you depending on what you ask. Below are concrete prompts — the ones to avoid, and the ones to use instead.

❌ DON'T	✅ DO
`"Solve this exercise for me."`	`"I tried this. Give me a hint, not the answer."`
Copy-paste the AI answer	`"Explain this concept like I'm new to it."`
Use Copilot / Cursor autocomplete	`"Make me a 5-question quiz on this lecture."`
Let AI write code you don't understand	`"Why did my command fail? Walk me through the error."`

"Ask for hints, not answers. Ask for explanations, not code."

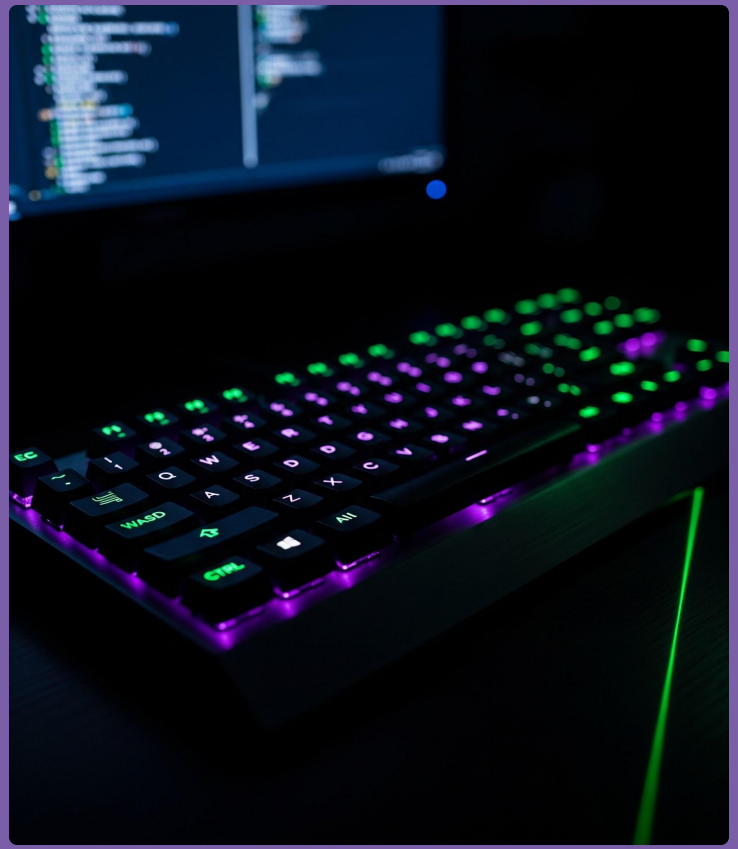
THE GOLDEN TEST

One simple rule separates students who grow from students who plateau. Apply it every single time you accept code from an AI.

After AI gives you code → delete it .

Try to rewrite it from memory. Can't? You don't understand it. Read again, line by line.

Tool	Policy during exercises
Copilot / Cursor	OFF
AI Chat (DeepSeek, ChatGPT, Claude...)	OK — only for hints, explanations, quizzes



"Type every character yourself. Yes it's slower. That's exactly the point."

ENGLISH — ON PURPOSE

All written materials in this course are in English.

Struggling with English here is good — you learn coding and English at the same time, and once you know the technical words, you can read any documentation in the world.

Career goal.

By end of year 3, you can talk about your code in English with a recruiter. That opens doors most of your peers cannot open.

"Two skills, one course. Embrace the discomfort — it's the price of the doors it opens."

HOW YOU'RE GRADED

Four moments in the semester decide your grade. Two projects, two exams. Plus bonus points for showing up and trying — even if your work is imperfect.

Evaluation Item	Weight	When
Solo project	20%	Sessions 10–12
Midterm exam	30%	Session 13
Group project	20%	Sessions 18–21
Final exam	30%	Session 24

Bonus points for regular exercise attempts (even imperfect, even half-finished) and class engagement.
Honest imperfect work beats polished AI submissions.

"Show me you tried. That's worth more than a perfect answer you can't explain."

BIG-O — THE FIVE TO REMEMBER

Big-O answers ONE question: how does this algorithm behave when the input gets really, really big? It does not measure seconds. It measures **growth**. Five classes will cover everything we do this semester. Memorise them.

[01] One loop over n → O(n). Linear.
[02] Two nested loops over n → O(n^2). Quadratic.
[03] Work cut in half each step → O(log n). Logarithmic.

Notation	Name	Typical example
O(1)	constant	`lst[i]`, `d[k]` lookup in a dict
O(log n)	logarithmic	Binary search, search in a balanced BST
O(n)	linear	Naive search, loop over a list once
O(n log n)	quasi-linear	Merge sort, quick sort (average case)
O(n^2)	quadratic	Bubble sort, two nested loops over the same list

"When you read code, count the loops. That's 80% of Big-O reading right there."

THE SPEED GAP — IT'S HUGE

Same problem. Same input. Different complexity = wildly different speed. On a list of one million items, a different algorithm makes the difference between a finger snap and 30 hours of compute. This is why algorithm choice matters — every day, at ByteDance, Alibaba, Tencent.

Algorithm	Complexity	Steps on 1,000,000 items
Access `lst[500_000]`	O(1)	**1**
Binary search	O(log n)	**~20**
Naive search	O(n)	**up to 1,000,000**
Bubble sort	$O(n^2)$	**~10^{12} (≈ 30 hours)**

20 vs 1,000,000. Same problem. Same input. **50,000× faster** — purely by choosing the right algorithm. You'll see this live in your terminal in today's practice.

"Algorithm choice is not optimisation theatre. It is the difference between 20 steps and 30 hours."

FIRST CLONE — GITEE + GIT

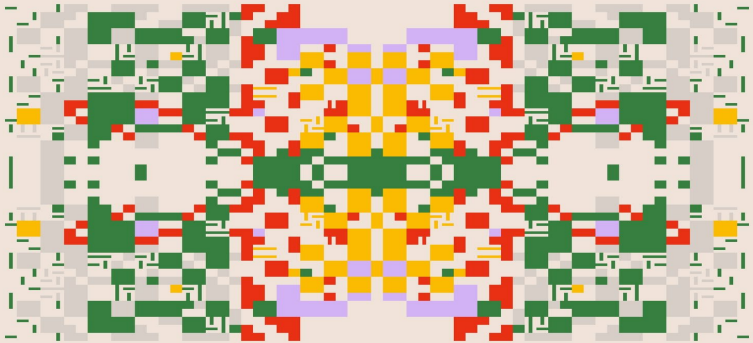
Three small steps to grab the course repo. **(1)** Open **gitee.com**, find the course repo. **(2)** Check if Git is on your machine. **(3)** Clone the repo to your Desktop. Same idea on Mac and Windows.

macOS (Terminal)

```
# 1. Is git already installed?
git --version

# 2. follow instructions on terminal

# 3. Clone to Desktop
cd ~/Desktop
git clone
https://gitee.com/adnan-boudjelal/algo_data-II.git
cd algo_data-II
ls
```

Windows (PowerShell)

```
# 1. Is git installed?
git --version

# 2. If not, install with winget
winget install --id Git.Git -e

# 3. Clone to Desktop
cd $HOME\Desktop
git clone
https://gitee.com/adnan-boudjelal/algo_data-II.git
cd algo_data-II
ls
```



EXERCICE

“Let’s pull today’s exercice and start to practice !”

▪ RECAP — AND WHAT'S NEXT ▪

TODAY, IN FIVE LINES

- **ALGO II** = trees, recursion, sorting, dynamic programming.
- The next 3 classes are a **review of ALGO I** — only the parts we'll need.
- The course is built around **practice**. Lecture is the appetizer; exercises are the meal.
- **AI = tutor, not solver.** Autocomplete OFF during exercises.
- **20 steps vs 200,000 steps.** Same problem. That's why algorithm choice matters.

THE ROAD AHEAD

Now → philosophy + Big-O wake-up

Next (S2) → Stacks, linked lists — the walking pattern

Then (S3) → Hash tables + first taste of recursion

After that (S4+) → Trees, BST, heaps, recursion deep dive

"Next class: stacks and linked lists. Two structures you already know — but with new eyes. The way you walk a linked list is the exact same pattern you'll use to walk a tree two weeks from now. Same reflex, just one extra pointer."

STACKS & LISTS → WALKING THE CHAIN

"Today we set the rules and felt the speed gap. Next class — we walk the chain."